

Relatório 1: Complexidade Espacial e Temporal

Análise de Algoritmos - Grupo 5 - Universidade Católica de Brasília

Implementação e análise de algoritmos recursivos e iterativos

Objetivo e procedimento experimental

Foram implementadas versões iterativas e recursivas para três problemas: contador de zeros em matriz, inversão de string e maior ocorrência de uma mesma letra em uma string. Para cada implementação foram analisadas as complexidades temporal e espacial, com foco no consumo da pilha, e os resultados teóricos foram comparados com medições experimentais.

Cada programa recebe pela linha de comando o valor de N e o nome do arquivo CSV. Para cada N, o algoritmo é executado 32 vezes e o tempo registrado é a média das execuções. O CSV armazena n, time e stack. Os dados e os códigos completos estão no repositório público:

<https://github.com/branchescunha/algorithm-analysis>

1. Contador de Zeros em Matriz

A entrada é uma matriz N x N. A versão iterativa percorre todas as posições com dois laços. A versão recursiva processa uma linha por chamada e percorre suas N colunas.

1.1 Implementação iterativa

```
long long count_zeros_iterative(const int *matrix, int n) {
    long long count = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (matrix[(size_t)i * n + j] == 0) count++;
    return count;
}
```

Complexidade temporal - $O(N^2)$: Se $E(N)=N^2$ é o número de elementos visitados, no caso base $E(1)=1$. Supondo $E(N)=N^2$, uma matriz $(N+1) \times (N+1)$ possui $N^2+2N+1=(N+1)^2$ posições. Logo o número de operações cresce quadraticamente.

Complexidade espacial da pilha - $O(1)$: A função não cria chamadas recursivas. Se $P(N)=c$ no caso base, para $N+1$ continua sendo $P(N+1)=c$. Portanto o consumo da pilha não depende de N.

1.2 Implementação recursiva

```
long long count_zeros_recursive(const int *matrix, int n, int row) {
    if (row == n) return 0;
    long long count = 0;
    for (int column = 0; column < n; column++)
        if (matrix[(size_t)row * n + column] == 0) count++;
    return count + count_zeros_recursive(matrix, n, row + 1);
}
```

Complexidade temporal - $O(N^2)$: Cada chamada trata uma linha com N elementos e existem N linhas. Pela recorrência $T(N)=N \cdot c + T_{\text{linhas}}(N-1)$, o total é proporcional a $N \cdot N$. Equivalentemente, são visitadas N^2 posições, logo $O(N^2)$.

Complexidade espacial da pilha - $O(N)$: Existe uma chamada recursiva por linha. Caso base $P(1)=c$. Supondo $P(N)=cN+b$, para $N+1$ ocorre uma chamada adicional: $P(N+1)=P(N)+c=c(N+1)+b$. Logo a pilha cresce linearmente.

N	Tempo it. (ms)	Pilha it. (B)	Tempo rec. (ms)	Pilha rec. (B)
100	0.031250	80	0.000000	6480
400	0.250000	80	0.718750	25680
700	0.562500	80	2.468750	44880
1000	1.250000	80	1.281250	64080

Comparação teórico x experimental: a pilha iterativa permaneceu em 80 bytes em todos os testes, enquanto a recursiva passou de 6.480 bytes em N=100 para 64.080 bytes em N=1000, confirmando $O(1)$ e $O(N)$, respectivamente. Os tempos apresentam oscilações pela resolução do clock(), mas aumentam com o crescimento da matriz e são compatíveis com o custo $O(N^2)$.

2. Inversor de String

Para uma string com N caracteres, a versão iterativa copia os caracteres da última posição para a primeira. A versão recursiva produz um caractere por chamada.

2.1 Implementação iterativa

```
int reverse_iterative(const char *input, char *output, int n) {
    for (int i = 0; i < n; i++)
        output[i] = input[n - 1 - i];
    output[n] = '\0';
    return n;
}
```

Complexidade temporal - $O(N)$: Se $I(N)=N$ iterações, $I(1)=1$. Assumindo $I(N)=N$, para $N+1$ é necessária uma iteração adicional: $I(N+1)=I(N)+1=N+1$. Logo o tempo é linear.

Complexidade espacial da pilha - $O(1)$: Não há novas chamadas de função durante o laço. Assim $P(N)=c$ e $P(N+1)=c$, independentemente do tamanho da string.

2.2 Implementação recursiva

```
int reverse_recursive(const char *input, char *output, int n, int pos) {
    if (pos == n) { output[pos] = '\0'; return 0; }
    output[pos] = input[n - 1 - pos];
    return 1 + reverse_recursive(input, output, n, pos + 1);
}
```

Complexidade temporal - $O(N)$: A recorrência é $T(N)=T(N-1)+c$. No caso base $T(0)=c_0$. Assumindo $T(N) \leq aN+b$, então $T(N+1) \leq aN+b+c \leq a(N+1)+b$ para $a \geq c$. Portanto $T(N)=O(N)$.

Complexidade espacial da pilha - $O(N)$: Cada caractere gera uma chamada. Se $P(N)=aN+b$, então $P(N+1)=P(N)+a=a(N+1)+b$. A profundidade cresce proporcionalmente a N.

N	Tempo it. (ms)	Pilha it. (B)	Tempo rec. (ms)	Pilha rec. (B)
1000	0.000000	80	0.000000	48064
4000	0.000000	80	0.031250	192064
7000	0.031250	80	0.000000	336064
10000	0.000000	80	0.156250	480064

Comparação teórico x experimental: a pilha iterativa permaneceu em 80 bytes. Na versão recursiva, cresceu de 48.064 bytes em N=1000 para 480.064 bytes em N=10000, evidenciando $O(N)$. Os tempos ficaram muito baixos e vários pontos aparecem como zero por limite de resolução do clock(); por isso, neste caso, a tendência espacial é mais nítida que a temporal.

3. Maior Ocorrência de uma Mesma Letra

As duas versões utilizam um vetor de 26 posições para contar letras. Na iterativa, a string é percorrida por um laço. Na recursiva, cada chamada contabiliza um caractere e avança uma posição.

3.1 Implementação iterativa

```
int most_frequent_iterative(const char *text, int n) {
    int frequency[26] = {0};
    for (int i = 0; i < n; i++) frequency[text[i] - 'a']++;
    int max = 0;
    for (int i = 0; i < 26; i++) if (frequency[i] > max) max = frequency[i];
    return max;
}
```

Complexidade temporal - $O(N)$: O primeiro laço percorre N caracteres e o segundo percorre sempre 26 posições. Assim $T(N)=aN+26b$. Se $T(N)\leq cN+d$, para $N+1$ adiciona-se apenas custo constante; logo permanece $O(N)$.

Complexidade espacial da pilha - $O(1)$: O vetor frequency possui sempre 26 inteiros, independentemente de N , e não há recursão. Logo o espaço adicional na pilha é constante.

3.2 Implementação recursiva

```
int count_recursive(const char *text, int n, int pos, int frequency[26]) {
    if (pos == n) return 0;
    frequency[text[pos] - 'a']++;
    return 1 + count_recursive(text, n, pos + 1, frequency);
}
```

Complexidade temporal - $O(N)$: A contagem satisfaz $T(N)=T(N-1)+c$ e, após a recursão, há apenas um laço constante de 26 posições. Pelo mesmo passo indutivo da inversão de string, $T(N)=O(N)$.

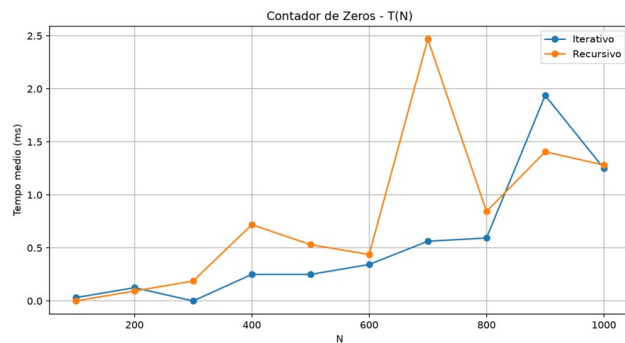
Complexidade espacial da pilha - $O(N)$: Cada posição da string adiciona um frame à pilha. Caso base $P(0)=b$; supondo $P(N)=aN+b$, então $P(N+1)=P(N)+a=a(N+1)+b$. Portanto $O(N)$.

N	Tempo it. (ms)	Pilha it. (B)	Tempo rec. (ms)	Pilha rec. (B)
1000	0.000000	208	0.031250	48240
4000	0.000000	208	0.093750	192240
7000	0.000000	208	0.062500	336240
10000	0.031250	208	0.031250	480240

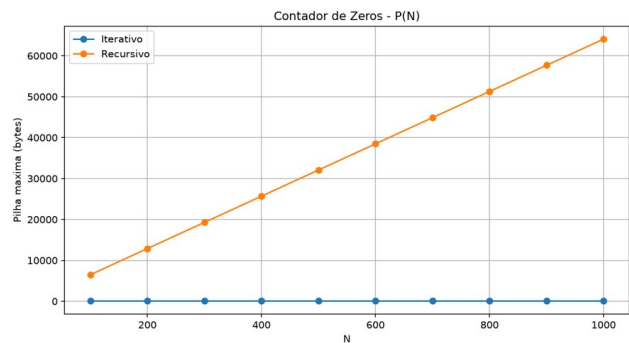
Comparação teórico x experimental: a versão iterativa permaneceu em 208 bytes, enquanto a recursiva passou de 48.240 bytes em $N=1000$ para 480.240 bytes em $N=10000$. Isso confirma $O(1)$ contra $O(N)$ para a pilha. O tempo também apresenta ruído de medição por ser muito pequeno, mas a análise teórica continua linear nas duas versões.

4. Gráficos dos Experimentos

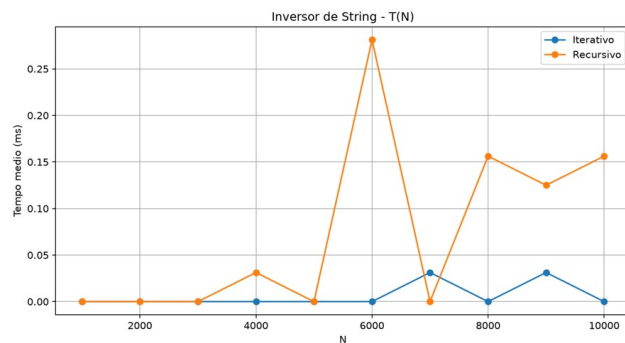
Os seis gráficos abaixo relacionam N com o tempo médio $T(N)$ e com o consumo máximo da pilha $P(N)$, comparando as versões iterativa e recursiva.



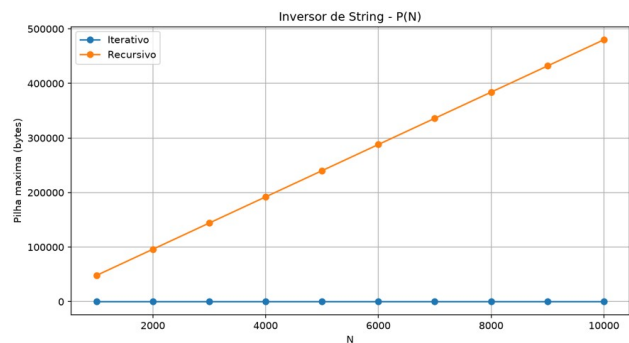
Zeros - T(N)



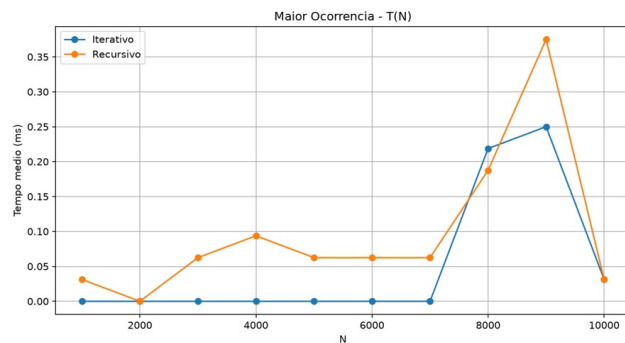
Zeros - P(N)



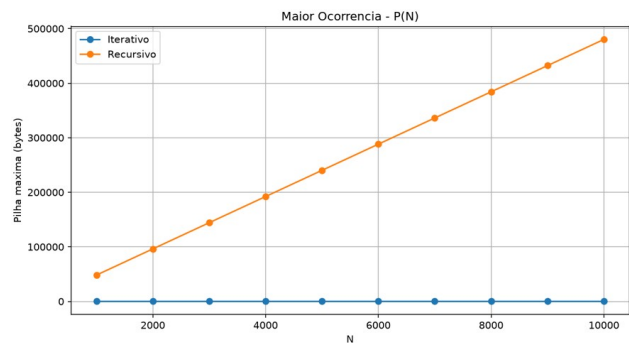
Inverso - T(N)



Inverso - P(N)



Maior ocorrência - T(N)



Maior ocorrência - P(N)

5. Conclusão

Os experimentos ficaram coerentes com a análise teórica. No contador de zeros, as duas versões têm tempo $O(N^2)$, mas a versão recursiva usa pilha $O(N)$, enquanto a iterativa usa $O(1)$. Nos dois problemas de string, as duas abordagens têm tempo $O(N)$; novamente, as versões iterativas mantêm pilha $O(1)$ e as recursivas apresentam $O(N)$. Assim, a recursão não alterou a ordem temporal nesses problemas, mas aumentou o consumo da pilha. As oscilações e valores zero observados nos tempos mais curtos são uma limitação da resolução do clock() e não contradizem as complexidades demonstradas.